

BST: Inorder Predecessor & Successor

Core Concept, Algorithm Logic, Step-by-Step Visual Walkthrough, and VS Code C++ Source Code

Binary Search Tree

Inorder Traversal

Time: O(h)

Space: O(1)

Interview Core

1. Overview & Key Applications

This is one of the most important Binary Search Tree (BST) concepts. It forms the foundation for tree navigation, deletion, and search algorithms.

LeetCode 285: Inorder Successor in BST

LeetCode 510: Inorder Successor II

LeetCode 450: Delete Node in BST

Ceil / Floor: Problems in Binary Search Trees

Top Technical Interviews: Amazon, Microsoft, Google, Adobe, Atlassian, Uber, Meta, etc.

STEP 1

What is Inorder Traversal?

Remember the fundamental traversal order:

Left Subtree → Root Node → Right Subtree

Sample BST:

```
      50
     /  \
    /    \
   /      \
  20  40  60  80
   \  /
    30  70
```

Inorder Traversal Output:

20 → 30 → 40 → 50 → 60 → 70 → 80

Notice something? The numbers are in **strictly sorted order**. Everything about predecessor and successor comes directly from this sorted sequence.

STEP 2

Inorder Predecessor

Definition: The node that appears *immediately before* a given node in the inorder traversal sequence.

Inorder: 20 → 30 → 40 → 50 → 60 → 70 → 80

• Suppose node = **50**:

20 → 30 → 40 → **[50]** → 60 → 70 → 80
Immediately before 50 is 40.

→ **Predecessor(50) = 40**

• Suppose node = **70**:

20 → 30 → 40 → 50 → 60 → **[70]** → 80
Previous element is 60.

→ **Predecessor(70) = 60**

STEP 3

Inorder Successor

Definition: The node that appears *immediately after* a given node in the inorder traversal sequence.

Inorder: 20 → 30 → 40 → 50 → 60 → 70 → 80

• Suppose node = **50**:

20 → 30 → 40 → **[50]** → 60 → 70 → 80
Immediately after 50 is 60.

→ **Successor(50) = 60**

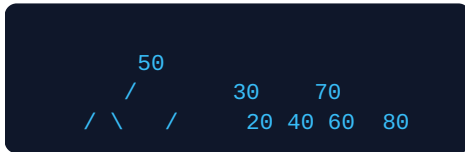
• Suppose node = **30**:

20 → **[30]** → 40 → 50 → 60 → 70 → 80
Next element is 40.

→ **Successor(30) = 40**

STEP 4

Visual Mapping Table for Every Node



Node	Inorder Predecessor	Inorder Successor
20	NULL	30
30	20	40
40	30	50
50	40	60
60	50	70
70	60	80
80	70	NULL

STEP 5

The Main Logic & Algorithms

This is where many students get confused. There are two distinct structural cases for both predecessor and successor.

FINDING INORDER PREDECESSOR

Case 1: Left Subtree Exists

Largest value before current node is in its left subtree.

Rule: Go **Left** once → Keep going **Right** until possible.

```
50 (target)
/
30 → Go Left
   40 → Go Right (Answer = 40)
```

Why? The rightmost node of the left subtree is the largest number smaller than current node.

Example (Node 80):

80 → Left(60) → Right(70) → Right(75)
→ **Predecessor(80) = 75**

Case 2: Left Child Does NOT Exist

No left child. Need to traverse down from root.

Algorithm: Whenever `current->val < target->val`, store current as predecessor, then move **Right**. Else move **Left**.

```
50 (Current=50 < 60 → pred=50, move Right)
70 (Current=70 > 60 → move Left)
/
60 (Current=60 == 60 → Stop)
```

Why? The stored node is the deepest ancestor where we turned right (the last node smaller than target).

→ **Predecessor(60) = 50**

FINDING INORDER SUCCESSOR

Case 1: Right Subtree Exists

Smallest value after current node is in its right subtree.

Rule: Go **Right** once → Keep going **Left** until possible.

```
50 (target)
  70 → Go Right
/
60 → Go Left
/
55 → Go Left (Answer = 55)
```

Why? The leftmost node of the right subtree is the smallest value larger than current node.

Case 2: Right Child Does NOT Exist

No right child. Traverse down from root.

Algorithm: Whenever `current->val > target->val`, store current as successor, then move **Left**. Else move **Right**.

```
50 (Current=50 > 40 → succ=50, move Left)
/
30 (Current=30 < 40 → move Right)
  40 (Current=40 == 40 → Stop)
```

Why? The stored node is the deepest ancestor where we turned left (the last node larger than target).

→ **Successor(40) = 50**

Why These Rules Work (BST Property: $\text{Left} < \text{Root} < \text{Right}$):

- Inside Left Subtree: Everything is smaller. The largest among them is the **Rightmost node**.
- Inside Right Subtree: Everything is larger. The smallest among them is the **Leftmost node**.

Algorithm Summary

PREDECESSOR SUMMARY

```
If Left Subtree Exists:
  Go Left once
  Keep Going Right till end

Else (No Left Subtree):
  Start from root
  if current < target:
    predecessor = current
    move right
  else:
    move left
```

SUCCESSOR SUMMARY

```
If Right Subtree Exists:
  Go Right once
  Keep Going Left till end

Else (No Right Subtree):
  Start from root
  if current > target:
    successor = current
    move left
  else:
    move right
```

C++ Implementation (VS Code Format)

Time: $O(h)$ **Space: $O(1)$** where h is the height of the Binary Search Tree.

inorder_predecessor.cpp

```
TreeNode* inorderPredecessor(TreeNode* root, TreeNode* p) {
    TreeNode* pred = nullptr;

    while (root) {
        if (p->val > root->val) {
            pred = root;
            root = root->right;
        } else {
            root = root->left;
        }
    }

    return pred;
}
```

inorder_successor.cpp

```
TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {
    TreeNode* succ = nullptr;

    while (root) {
        if (p->val < root->val) {
            succ = root;
            root = root->left;
        } else {
            root = root->right;
        }
    }

    return succ;
}
```

Complete Version (Explicit Subtree vs Fallback Handling)

complete_predecessor.cpp

```
TreeNode* inorderPredecessor(TreeNode* root, TreeNode* p) {
    // Case 1: Left subtree exists
    if (p->left) {
        TreeNode* cur = p->left;
        while (cur->right)
            cur = cur->right;
        return cur;
    }

    // Case 2: No left subtree
    TreeNode* pred = nullptr;

    while (root) {
        if (p->val > root->val) {
            pred = root;
            root = root->right;
        } else {
            root = root->left;
        }
    }

    return pred;
}
```

complete_successor.cpp

```
TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {
    // Case 1: Right subtree exists
    if (p->right) {
        TreeNode* cur = p->right;
        while (cur->left)
            cur = cur->left;
        return cur;
    }

    // Case 2: No right subtree
    TreeNode* succ = nullptr;

    while (root) {
        if (p->val < root->val) {
            succ = root;
            root = root->left;
        } else {
            root = root->right;
        }
    }

    return succ;
}
```

⚡ EASY MEMORY TRICK

PREDECESSOR

- Left once → Right till end
- (No left?) Remember last **Smaller** node.

SUCCESSOR

- Right once → Left till end
- (No right?) Remember last **Greater** node.

💡 One-Line Intuition

Imagine the BST's inorder traversal as a flattened, sorted array:

20 → 30 → 40 → 50 → 60 → 70 → 80

- **Predecessor** = the previous element in this sorted order.
- **Successor** = the next element in this sorted order.

The BST algorithms above simply find those neighbors directly using tree structure without actually building the sorted array, which is why they run in $O(h)$ time instead of $O(n)$. This viewpoint makes it much easier to understand why we go to the rightmost node of the left subtree for the predecessor and the leftmost node of the right subtree for the successor.